Contents

Chapter 10: Reliability Engineering

AI systems are probabilistic systems embedded inside deterministic infrastructure.

That distinction makes reliability engineering more important—not less.

A traditional distributed system may fail because a server crashes, a network connection drops, a database becomes unavailable, or a dependency times out. An AI system can fail in all of those ways **and** fail while everything is technically operating correctly.

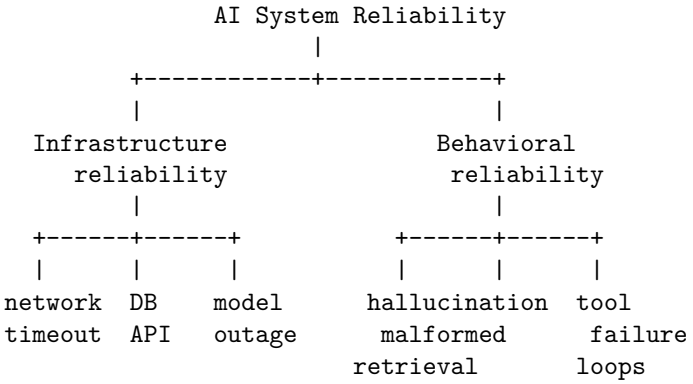
The API can return HTTP 200 and the model can produce an answer that is completely wrong.

A retrieval system can return documents successfully—but the wrong documents.

A tool can execute successfully—but return data that causes the agent to make a bad decision.

An agent can follow its instructions perfectly and still enter an infinite loop.

Reliability engineering for AI therefore has two dimensions:



The goal is not to build a system that never fails.

That system does not exist.

The goal is to build a system whose failures are:

- bounded
- detectable
- recoverable
- explainable
- observable
- safe

The fundamental engineering question is:

> **What happens if every component fails?**

Not whether it fails.

What happens when it fails?

---

1. Reliability Is a System Property

A common mistake is to think about reliability component-by-component.

For example:

> "Our model API has 99.9% availability."

That tells us almost nothing about the reliability of the application.

Suppose an AI application contains:

```
Frontend
   ↓
```

```
API
   ↓
Agent
   ↓
LLM
   ↓
Retriever
   ↓
Vector database
   ↓
Tool API
   ↓
Transactional database
```

If these components are independent, the availability of the complete synchronous path is approximately:

$$A_{system} = \prod_i A_i$$

Ten components with 99.9% availability each produce:

$$0.999^{10} \approx 99.0\%$$

The application has approximately **1% downtime**, despite every individual dependency having an apparently excellent 99.9% availability.

And this calculation is optimistic.

Real systems contain:

- correlated failures
- cascading failures
- overloaded dependencies
- retries
- queues
- shared infrastructure
- regional outages
- configuration errors
- deployment failures
- resource exhaustion

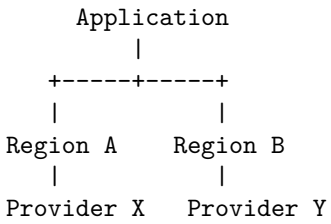
Reliability must therefore be designed at the **system level**.

---

2. Failure Domains

A failure domain is a set of components that can fail together.

Consider:



If the entire application depends on one cloud region, distributing services across multiple availability zones may not protect against a regional outage.

Similarly:

```
LLM
 |
 +-- API endpoint
 +-- authentication
 +-- network
 +-- provider infrastructure
```

These may appear to be separate dependencies but can share a common failure domain.

For AI applications, important failure domains include:

- process
- host
- container
- availability zone
- region
- cloud provider
- model provider
- model version
- database
- retrieval infrastructure
- external APIs
- credentials
- network
- human operators
- configuration/deployment

The engineering question becomes:

> **What is the largest failure domain this architecture can tolerate?**

A highly available application should explicitly define its failure assumptions.

For example:

```
Can tolerate:
+ single process failure
+ single container failure
+ single AZ failure
+ transient LLM failure

Cannot tolerate:
x complete cloud-region failure
x identity provider failure
x corruption of primary database
```

That is a much more useful reliability specification than simply saying "high availability."

---

3. Timeouts

Every external operation needs a timeout.

Without one, a failed dependency can consume resources indefinitely.

Consider:

```
response = llm.generate(request)
```

If the provider becomes unresponsive, the request may remain alive until infrastructure eventually kills it.

Now multiply this by thousands of concurrent requests.

The result can be resource exhaustion.

Timeouts establish a fundamental invariant:

$$T_{operation} \leq T_{max}$$

But timeout design is hierarchical.

Suppose:

```
User request
    30 sec
       |
       +-- retrieval: 3 sec
       |
       +-- LLM: 20 sec
```

```
      |
      +-- tools: 5 sec
```

The component budgets must fit inside the parent budget.

Otherwise:

```
API timeout = 30 sec

retrieval = 20 sec
LLM = 20 sec
tool = 20 sec
```

A theoretically sequential execution path could require 60 seconds.

Timeouts should therefore be treated as **budgets**, not arbitrary numbers.

---

4. Retries

Transient failures are common:

```
request
   ↓
timeout
   ↓
retry
   ↓
success
```

Retries can dramatically improve reliability.

But retries can also destroy a system.

Suppose 1,000 clients simultaneously encounter a dependency failure.

If every client immediately retries:

```
1,000 requests
      ↓
failure
      ↓
1,000 retries
      ↓
failure
      ↓
1,000 retries
```

The dependency receives a retry storm precisely when it is least capable of handling additional load.

This is the classic **retry amplification** problem.

Retries should therefore have:

- bounded attempt counts
- exponential backoff
- jitter
- appropriate retry classification
- total time budgets

A common strategy is:

$$t_n = \min(t_{max}, t_0 2^n)$$

with randomized jitter:

$$t'_n = U(0, t_n)$$

where U is a uniform random variable.

For example:

```
Attempt 1: 0.5 s
Attempt 2: 1 s
Attempt 3: 2 s
Attempt 4: 4 s
```

with jitter applied to avoid synchronization.

Do not retry everything A useful classification is:

```
Retry:
+ timeout
+ transient network error
+ 429
+ selected 5xx errors

Do not retry:
x invalid request
x authentication failure
x malformed input
x deterministic validation failure
```

AI systems add another dimension.

A malformed model response may be recoverable through:

```
LLM
 ↓
schema validation
 ↓
```

```
invalid
 ↓
repair / constrained retry
```

But blindly regenerating indefinitely is dangerous.

Every retry consumes:

- latency
- compute
- money
- context
- rate-limit capacity

---

5. Idempotency

Retries are safe only when repeated operations do not create unintended side effects.

Consider:

```
Agent → "Send payment of $500"
```

The request succeeds.

The response is lost.

The agent retries.

Now the user has paid $1,000.

The solution is **idempotency**.

An operation is idempotent when repeating it produces the same intended effect.

For example:

```
POST /payment

Idempotency-Key: 8f3a...
```

The server records the operation associated with that key.

A retry with the same key becomes:

```
same request
     ↓
same operation
     ↓
same result
```

This is particularly important for agents because agents naturally retry.

Agentic systems should distinguish:

```
READ operations
    ↓
usually safe to retry

WRITE operations
    ↓
require idempotency
```

Examples of dangerous non-idempotent actions include:

- sending email
- creating payments
- submitting orders
- modifying records
- deleting resources
- publishing messages
- invoking external actions

The agent should never be allowed to infer that "retry" means "perform the side effect again."

---

6. Circuit Breakers

Suppose an external service is failing continuously.

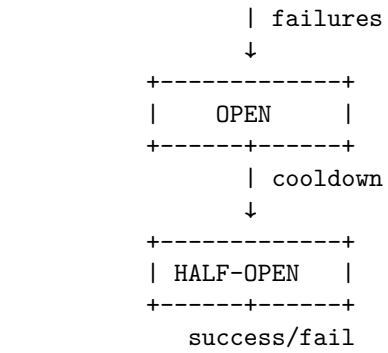
Without protection:

```
Application
     |
     +-- request
     +-- retry
     +-- retry
     +-- retry
     +-- retry
          ↓
       failing API
```

The application wastes resources attempting operations that are unlikely to succeed.

A circuit breaker changes this behavior.

```
          +-------------+
          |   CLOSED    |
          +------+------+
```



When the failure rate exceeds a threshold, the circuit opens.

Requests fail fast instead of reaching the unhealthy dependency.

This protects both systems.

For AI applications, circuit breakers are especially useful around:

- LLM providers
- embedding services
- vector databases
- external APIs
- expensive tools
- secondary model providers

---

7. Graceful Degradation

A reliable system does not necessarily provide its full functionality during failure.

It provides the **best safe functionality that remains possible**.

Suppose an AI research assistant depends on:

```
LLM
Retriever
Web search
Document store
```

If web search fails, the application might still answer from its local corpus.

If retrieval fails, it might refuse to answer rather than hallucinate.

If the primary model fails, it might switch to a fallback model.

This creates a degradation hierarchy:

```
Full functionality
       ↓
No web search
```

```
       ↓
Local retrieval only
       ↓
Cached responses
       ↓
Read-only mode
       ↓
Explicit unavailable state
```

The important distinction is:

Graceful degradation is not "make something up."

In AI systems, the safest fallback is often a more constrained behavior.

For example:

```
Retrieval unavailable
        ↓
Do not answer from unsupported knowledge
        ↓
Explain that evidence is unavailable
```

This is more reliable than producing a fluent but ungrounded answer.

---

8. Load Shedding

When demand exceeds capacity, something must give.

A system that attempts to serve everything can collapse completely.

Load shedding deliberately rejects or reduces work to preserve the core service.

For example:

```
Capacity = 1,000 req/s
Demand   = 2,000 req/s
```

Instead of allowing latency to grow without bound:

```
2,000 requests
      ↓
queue grows
      ↓
latency grows
      ↓
timeouts
      ↓
retries
      ↓
```

```
more load
      ↓
system collapse
```

the system can shed load:

```
2,000 requests
      ↓
1,000 accepted
1,000 rejected
      ↓
healthy system
```

AI applications have particularly strong incentives for load shedding because requests can have highly variable cost.

One request might require:

```
1 LLM call
```

while another requires:

```
planner
  ↓
retrieval
  ↓
5 tool calls
  ↓
3 LLM calls
  ↓
verification
  ↓
final response
```

The second request may consume orders of magnitude more resources.

Useful controls include:

- concurrency limits
- token budgets
- request quotas
- priority queues
- maximum agent steps
- maximum tool calls
- maximum context size
- per-user budgets
- deadline propagation

---

9. SLI, SLO, and Reliability Targets

Reliability needs measurable definitions.

An **SLI**—Service Level Indicator—is a measurement.

Examples:

```
availability
latency
error rate
tool-call success rate
retrieval success rate
```

An **SLO**—Service Level Objective—is a target.

For example:

```
99.9% of requests succeed
99% complete within 5 seconds
99.5% of tool calls return valid results
```

The distinction matters.

"Fast" is not an SLO.

"99% of interactive requests complete within 5 seconds" is.

AI systems require behavioral SLIs as well.

Traditional infrastructure metrics:

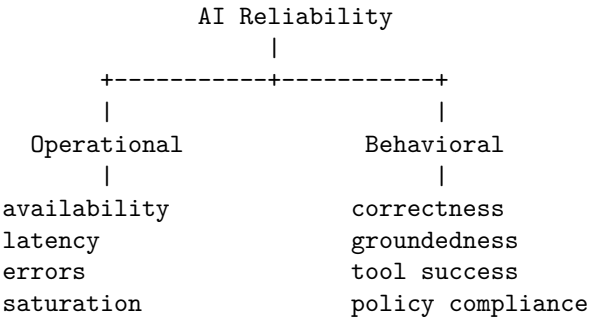
```
CPU
memory
latency
availability
errors
```

are insufficient.

AI-specific metrics might include:

```
schema-valid response rate
groundedness
citation correctness
tool-call success
retrieval recall
agent completion rate
unsafe-action rate
human escalation rate
```

The reliability dashboard therefore becomes:



This leads to an important principle:

> **An AI system can be operationally available and behaviorally unavailable.**

If the model answers every request but 20% of answers are materially incorrect, the service is not reliable from the user's perspective.

---

10. Disaster Recovery

Reliability engineering must consider failures larger than individual requests.

Disaster recovery asks:

> What happens when the system cannot operate normally?

Two classic metrics are:

RTO — Recovery Time Objective How quickly must the system be restored?

$$RTO = \text{maximum acceptable recovery time}$$

RPO — Recovery Point Objective How much data loss is acceptable?

$$RPO = \text{maximum acceptable data-loss window}$$

For example:

```
RTO = 1 hour
RPO = 5 minutes
```

means the system should recover within one hour while losing no more than five minutes of recent data.

AI systems introduce additional disaster-recovery concerns.

You may need to preserve:

- prompts
- system instructions
- model versions
- tool definitions
- retrieval indexes
- embeddings
- evaluation datasets
- agent state
- configuration
- secrets
- audit logs
- model routing policies

A database backup alone may not reproduce the system.

The **AI system is a combination of state, configuration, models, tools, and orchestration logic.**

---

11. AI-Specific Failure Modes

Traditional reliability engineering is necessary but insufficient.

AI systems have failure modes that are fundamentally behavioral.

Model unavailable

The provider returns:

```
timeout
503
429
authentication failure
```

Possible responses:

```
retry
   ↓
fallback model
   ↓
cached response
   ↓
degraded mode
   ↓
explicit failure
```

The system should not silently fabricate a response.

Model behavior changes

A model provider can change the model behind an API.

The API remains healthy.

Latency remains healthy.

HTTP status remains 200.

But behavior changes.

For example:

```
Before:
structured JSON → 99.8% valid

After:
structured JSON → 96.1% valid
```

Operational monitoring sees nothing wrong.

Behavioral evaluation does.

This is why production AI systems require **continuous evaluation**, not merely uptime monitoring.

Malformed output

LLMs generate strings.

Your application usually requires structure.

Therefore:

```
LLM
 ↓
parser
 ↓
schema validator
 ↓
application
```

must treat model output as untrusted input.

Never assume:

```
"the model was instructed to return JSON"
```

means:

```
"the model returned valid JSON"
```

Validation must occur at the boundary.

A robust pattern is:

```
Generate
   ↓
Parse
   ↓
Validate
   ↓
Repair/retry if appropriate
   ↓
Validate again
   ↓
Execute
```

The final execution layer should accept only validated data.

---

12. Hallucinations Are Reliability Failures

Hallucination is often discussed as a model-quality problem.

Operationally, it is a reliability problem.

Consider:

```
User asks factual question
        ↓
retrieval fails
        ↓
model receives insufficient evidence
        ↓
model generates plausible answer
        ↓
application presents it as fact
```

Every infrastructure component may have succeeded.

The system still failed.

A reliable AI architecture therefore needs **epistemic controls**.

Examples include:

- retrieval requirements
- citation requirements
- confidence thresholds
- abstention

- evidence validation
- source provenance
- answer verification
- human escalation

The system should have a legitimate state of:

```
"I don't have enough evidence to answer this."
```

Abstention is often a reliability feature.

---

13. Retrieval Failure

RAG systems introduce another failure domain.

Retrieval can fail in several ways:

```
No documents found
      |
Wrong documents
      |
Outdated documents
      |
Incomplete documents
      |
Conflicting documents
      |
Correct documents but wrong ranking
```

A successful vector database query does not imply successful retrieval.

For example:

```
retrieval latency: 50 ms
retrieval status: 200
documents returned: 5
```

Everything looks healthy.

But if none of the five documents answer the question, retrieval has failed semantically.

Therefore RAG systems require metrics beyond infrastructure health:

Recall@k

Precision@k

and ultimately:

Answer Accuracy

The reliability chain is:

```
query
 ↓
retrieval
 ↓
evidence quality
 ↓
generation
 ↓
grounded answer
```

Every stage can fail independently.

---

14. Context Overflow

Context is a finite resource.

An agent may accumulate:

```
system instructions
+ conversation
+ retrieved documents
+ tool outputs
+ previous reasoning
+ intermediate results
```

until the context window becomes saturated.

A naive system might simply fail.

A robust system treats context as a managed resource.

For example:

```
Context budget
      |
      +-- system instructions
      +-- current task
      +-- relevant history
      +-- retrieved evidence
      +-- tool results
```

When the budget becomes constrained, the system can:

- summarize history
- remove irrelevant messages
- compress tool output
- rerank retrieved evidence
- discard redundant context
- start a fresh agent state

Context management is therefore part of reliability engineering.

A system that works for a five-turn conversation but fails after 200 turns is not reliably designed.

---

15. The Runaway Agent

One of the most distinctive AI reliability failures is the runaway agent.

Consider:

```
Agent
 ↓
tool
 ↓
observe
 ↓
plan
 ↓
tool
 ↓
observe
 ↓
plan
 ↓
...
```

If the stopping condition is weak, the loop can continue indefinitely.

This creates:

- unbounded latency
- unbounded token consumption
- repeated side effects
- API exhaustion
- financial cost
- cascading failures

Agents therefore need explicit resource limits.

For example:

```
max_steps = 20
max_tool_calls = 30
max_tokens = 100,000
max_wall_time = 120 seconds
max_cost = $1
```

The system should enforce these limits **outside the model**.

Do not rely on:

> "The agent will know when it is done."

The agent is a probabilistic component.

The runtime must enforce termination.

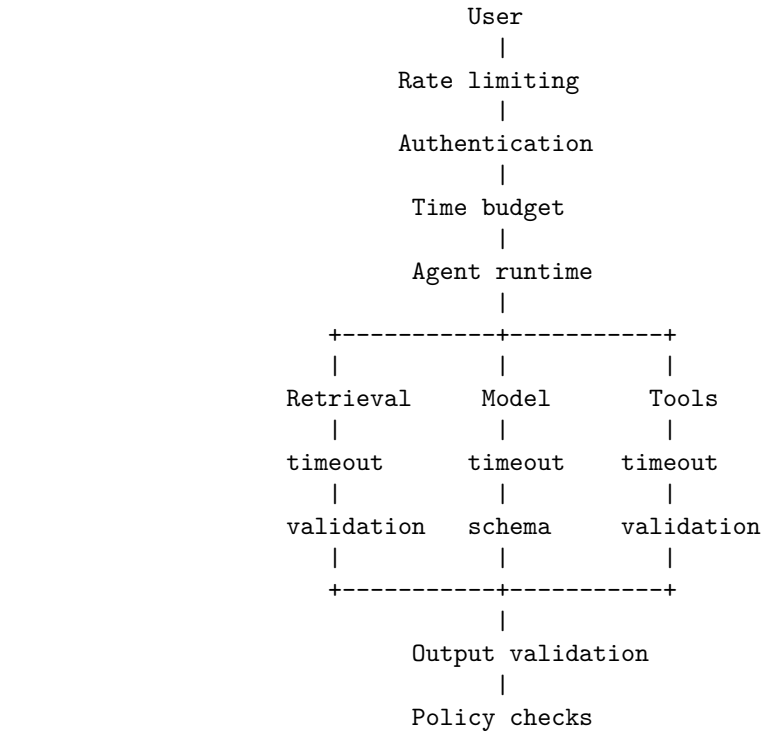
---

16. Reliability Through Defense in Depth

A mature AI system does not have one reliability mechanism.

It has layers.

Consider:



```
                    |
              Final response
```

Every layer assumes the layer below it can fail.

This is **defense in depth**.

The important architectural principle is:

> **Never rely on a single component to guarantee system correctness.**

The model should not be the security boundary.

The model should not be the reliability boundary.

The model should not be the authorization boundary.

The model should not be the termination boundary.

The deterministic runtime should enforce those properties.

---

17. Failure-Mode Design

One of the most useful exercises is to construct a failure-mode table.

Component	Failure	Detection	Recovery	Safe fallback
LLM	timeout	timeout metric	retry/fallback	degraded mode
LLM	malformed output	schema validation	repair/retry	reject
Retriever	unavailable	health check	retry/fallback	abstain
Retriever	bad results	retrieval eval	rerank/retry	abstain
Tool	timeout	deadline	retry	skip tool
Tool	bad result	validation	reject	continue safely
Agent	infinite loop	step counter	terminate	partial result
Context	overflow	token counter	compress	restart state
Database	unavailable	connection errors	failover	read-only mode
Provider	behavior drift	evals	model rollback	fallback model

This table turns vague reliability concerns into engineering requirements.

The same approach can be extended into a **failure-mode and effects analysis (FMEA)**.

For each failure, ask:

1. What can fail?
2. How will we detect it?
3. How will we recover?
4. What happens if recovery fails?
5. What is the safest degraded state?
6. Can the failure cascade?
7. What is the blast radius?

---

18. The Reliability Exercise

Take the Week 1 Personal Research Assistant.

Do not add new features.

Instead, attack it.

Assume:

```
LLM unavailable
Retriever unavailable
Vector database slow
Search API returns 429
Tool returns malformed JSON
Database connection drops
Model returns hallucinated citations
Context exceeds the limit
Agent enters an infinite loop
Traffic increases 10x
```

For each failure, determine:

```
Detection
   ↓
Containment
   ↓
Recovery
   ↓
Fallback
   ↓
User-visible behavior
```

For example:

Scenario: LLM timeout

```
LLM timeout
     ↓
deadline exceeded
```

```
     ↓
retry with backoff
     ↓
second failure
     ↓
fallback model
     ↓
fallback failure
     ↓
return explicit degraded response
```

Now ask:

> Does the system remain safe?

Then ask the harder question:

> Does the system remain useful?

Finally:

> Does the system remain economically bounded?

A reliable AI system must satisfy all three.

---

19. Reliability as a Specification

Reliability should not be something added after implementation.

It belongs in the specification.

Instead of:

> "The assistant should answer user questions."

write:

> The assistant shall return a response within 10 seconds for 99% of interactive requests under normal load.

Instead of:

> "The agent should use tools."

write:

> The runtime shall terminate an agent execution after 20 tool calls or 120 seconds, whichever occurs first.

Instead of:

> "The assistant should provide accurate answers."

write:

> Answers requiring external evidence shall contain validated citations to retrieved sources; if sufficient evidence cannot be retrieved, the assistant shall abstain rather than generate an unsupported factual claim.

Instead of:

> "The application should handle model failures."

write:

> If the primary model fails transiently, the runtime shall retry at most twice using exponential backoff. If all attempts fail, the request shall be routed to the configured fallback model or enter a degraded state.

These are **testable reliability requirements**.

That is the transition from:

```
Reliability as intention
```

to:

```
Reliability as specification
```

---

20. The Reliability Mindset

The deepest lesson is not any particular mechanism.

It is a change in how the engineer thinks.

The naive design question is:

> "How does the system work?"

The reliability engineer asks:

> "How does the system fail?"

Then:

> "How does it fail under load?"

Then:

> "How does it fail when a dependency fails?"

Then:

> "How does it fail when recovery fails?"

Then:

> "What happens when multiple things fail simultaneously?"

And finally:

> **"What is the worst thing this system can do while still believing it is operating normally?"**

For AI systems, that final question is particularly important.

A crashed server is obvious.

A hallucinating agent that successfully completes an action may not be.

A failed request is visible.

A plausible false answer may not be.

A timeout consumes resources.

A runaway agent can consume them indefinitely.

Reliability engineering therefore becomes the discipline of controlling both **failure probability** and **failure consequences**.

---

21. Key Takeaways

1. **Reliability is a system property, not a component property.** High availability of individual services does not guarantee high availability of the complete AI workflow.
2. **Design around failure domains.** Know which components can fail together and what the largest failure domain your architecture can tolerate.
3. **Every external operation needs a bounded timeout.** Timeouts are resource-protection mechanisms, not merely user-experience settings.
4. **Retries must be deliberate.** Use exponential backoff, jitter, bounded attempts, and retry classification. Blind retries can amplify outages.
5. **Retries require idempotency.** Particularly for agentic systems, distinguish safe reads from side-effecting writes.
6. **Circuit breakers prevent cascading failure.** Fail fast when a dependency is persistently unhealthy rather than repeatedly consuming resources.
7. **Graceful degradation is essential.** A degraded system should provide less functionality safely—not fabricate functionality it cannot support.
8. **Load shedding protects the system under overload.** AI workloads require explicit limits on concurrency, tokens, agent steps, tool calls, time, and cost.
9. **SLIs and SLOs must include behavioral quality.** Availability and latency are insufficient; AI systems also need metrics for groundedness, schema validity, retrieval quality, tool success, and task completion.

10. **Hallucinations are reliability failures.** A system that is operationally healthy but systematically produces unsupported answers is not reliable.
11. **Context is a finite reliability resource.** Context overflow, pollution, and uncontrolled accumulation must be managed explicitly.
12. **Agents require deterministic runtime limits.** Maximum steps, tool calls, tokens, wall time, and cost should be enforced outside the model.
13. **Disaster recovery includes AI state.** Backups must account for data, configuration, prompts, model versions, retrieval indexes, tools, evaluations, and orchestration state.
14. **Reliability belongs in the specification.** Requirements should define measurable behavior under normal operation, degraded operation, and failure.
15. **The fundamental reliability question is:**

    What happens if every component fails?

The objective is not to eliminate failure.

It is to make failure **bounded, observable, recoverable, and safe**.

And for AI systems, that means engineering not only for **service availability**, but for **behavioral integrity under failure**.